Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)
[Course Code: 23ECSPE1AD]

Submitted by

Suhasini Ganiger **1BM24EC221**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

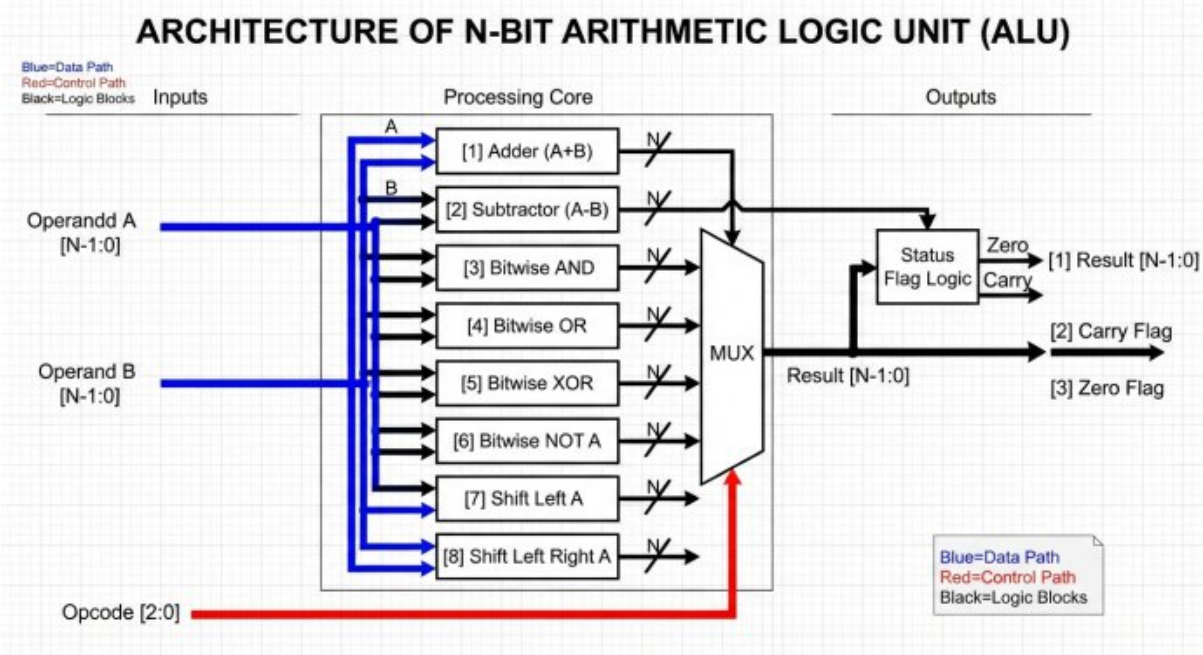
2026 – 2027

VERILOG CODES

1.ARITHEMATIC LOGIC UNIT

1. **Arithmetic Unit (AU):** Performs mathematical operations on binary numbers, including addition, subtraction, and sometimes multiplication or division. Subtraction is uniquely handled by adding the 2's complement of a number.
2. **Logic Unit (LU):** Performs bitwise Boolean operations on binary digits, such as AND, OR, NOT, XOR, and XNOR. It also handles **bit-shifting** (moving bits left or right), which is crucial for fast multiplication or data masking.

DESIGN



HDL CODE

```
module alu (
   input  [3:0] A,
   input  [3:0] B,
   input  [2:0] sel,
   output [3:0] Y,
   output Cout
);
wire [4:0] sum;
assign sum = {1'b0, A} + {1'b0, B};
assign Y = (sel == 3'b000) ? sum[3:0] :
        (sel == 3'b001) ? A - B :
```

```
        (sel == 3'b010) ? A & B :
        (sel == 3'b011) ? A | B :
        (sel == 3'b100) ? A ^ B :
        (sel == 3'b101) ? ~A :
        (sel == 3'b110) ? A << 1 :
                    A >> 1;
assign Cout = (sel == 3'b000) ? sum[4] : 1'b0;
```

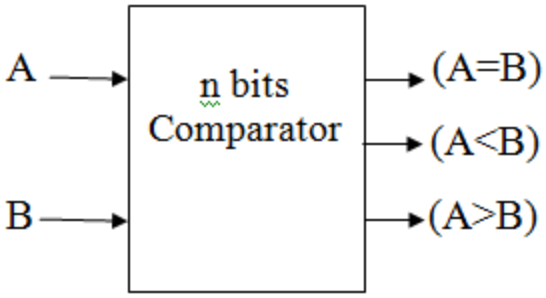
TESTBENCH

```
`timescale 1ns / 1ps
module alu_tb;
 reg  [3:0] A;
  reg  [3:0] B;
    reg  [2:0] sel;
wire [3:0] Y;
    wire Cout
// Instantiate the ALU
    alu uut( .A(A),.B(B) ,.sel(sel), .Y(Y), .Cout(Cout) );
 initial begin
  $monitor( "Time=%0t A=%b B=%b sel=%b Y=%b Cout=%b", $time, A, B, sel, Y, Cout );
  // Addition
       A = 4'b0101;       B = 4'b0011;      sel = 3'b000;         #10;
  // Subtraction
       A = 4'b1000;       B = 4'b0010;       sel = 3'b001;       #10;
// AND
       A = 4'b1100; B = 4'b1010;       sel = 3'b010;       #10;
  // OR
  A = 4'b1100;       B = 4'b1010;       sel = 3'b011;   10;
  // NOT
       A = 4'b1010;       B = 4'b0000;  sel = 3'b101;       #10;
  // Left shift
       A = 4'b0011;        B = 4'b0000;  sel = 3'b110;        #10;
```

```
// Right shift
      A = 4'b1100;      B = 4'b0000;      sel = 3'b111;      #10;
      $finish
 end
endmodule
```

1. COMPARATOR

A comparator is a combinational digital circuit that compares two binary numbers and determines whether the first number is greater than, less than, or equal to the second number.



HDL CODE

```
module comparator (
   input  [3:0] A,
   input  [3:0] B,
   output A_gt_B,
   output A_eq_B,
   output A_lt_B
);

   assign A_gt_B = (A > B);
   assign A_eq_B = (A == B);
   assign A_lt_B = (A < B);
endmodule
```

TESTBENCH

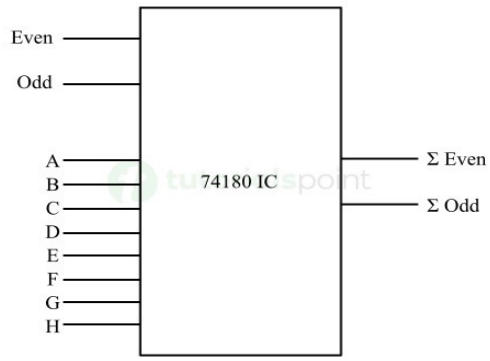
`timescale 1ns / 1ps

```
module comparator_tb;
```

```
    reg [3:0] A;
    reg [3:0] B;
    wire A_gt_B;
    wire A_eq_B;
    wire A_lt_B;
    // Instantiate the comparator
    comparator uut
(   .A(A),  .B(B), .A_gt_B(A_gt_B),  .A_eq_B(A_eq_B), .A_lt_B(A_lt_B));
initial begin
$monitor(
          "Time=%0t A=%b B=%b | A>B=%b A=B=%b A<B=%b", $time, A, B, A_gt_B,
A_eq_B, A_lt_B );
// Test 1: A > B
       A = 4'b1010; B = 4'b0110;   #10;
 // Test 2: A < B
         A = 4'b0011; B = 4'b1001;   #10;
   // Test 3: A = B
       A = 4'b1100; B = 4'b1100;   #10;
     $finish;
end
endmodule
```

2. 4-BIT PARITY GENERATOR AND CHECKER

A parity generator generates a parity bit to detect errors in binary data. It uses XOR gates to generate even or odd parity for a 4-bit input. A parity checker checks the received data along with its parity bit. If the parity is incorrect, an error is detected



HDL CODE

```
module parity_gen (
   input  [3:0] A,
   output P_even,
   output P_odd
);
assign P_even = A[3] ^ A[2] ^ A[1] ^ A[0];
   assign P_odd  = ~(A[3] ^ A[2] ^ A[1] ^ A[0]);
endmodule
```

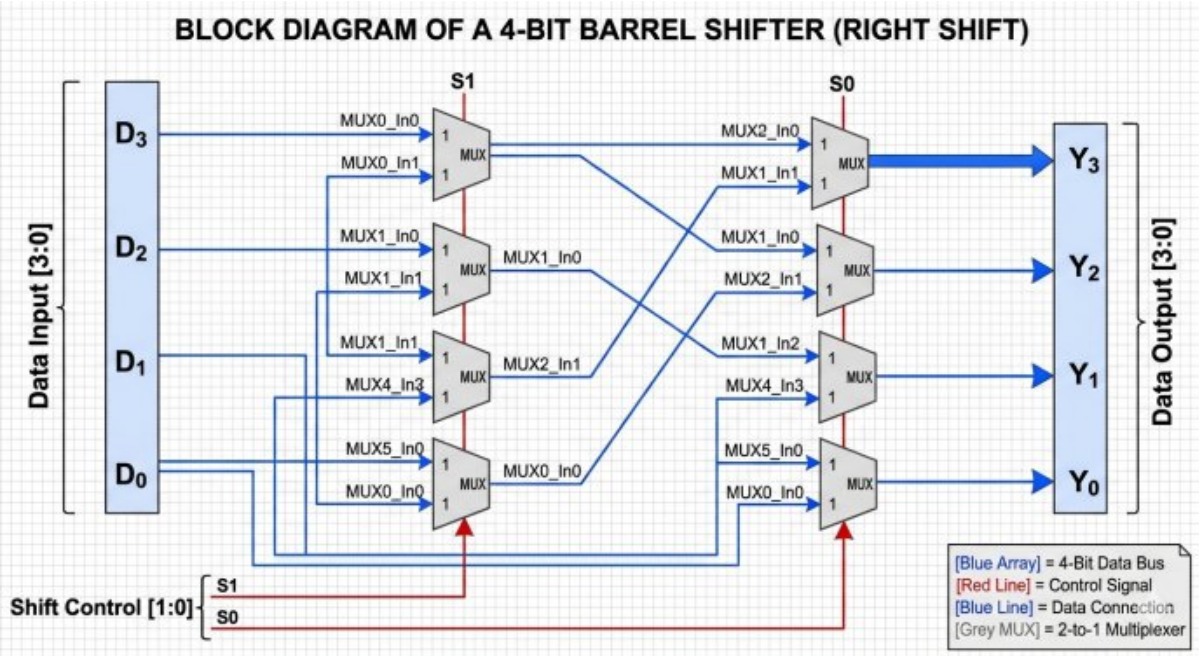
TESTBENCH

```
`timescale 1ns / 1ps
module parity_gen_tb;
 reg  [3:0] A;
   wire P_even;
   wire P_odd;
 // Instantiate the parity generator
   parity_gen uut ( .A(A) .P_even(P_even), .P_odd(P_odd)
  initial begin
      $monitor( "Time=%0t A=%b P_even=%b P_odd=%b",  $time, A, P_even, P_odd);
  A = 4'b0000; #10;
   A = 4'b000; #10;
A = 4'b0011; #10;
$finish;
end
endmodule
```

3. 4 BIT BARREL SHIFTER

A 4-bit barrel shifter is a combinational digital circuit that shifts the bits of a 4-bit binary input to the left or right by a specified number of positions in a single operation. Unlike a basic shifter that shifts one position at a time, a barrel shifter can shift multiple positions without requiring repeated clock cycles.

DESIGN



HDL CODE

```
module barrel_shifter (
   input  [3:0] A,
   input  [1:0] shift,
   input        dir,
   output [3:0] Y);
 assign Y = (dir == 1'b0) ? (A << shift) :(A >> shift);
endmodule
```

TEST BENCH

```
`timescale 1ns / 1ps
module barrel_shifter_tb;
   reg  [3:0] A;
   reg  [1:0] shift;
   reg        dir;
```

```
    wire [3:0] Y;
// Instantiate the barrel shifter
    barrel_shifter uut ( .A(A),  .shift(shift), .dir(dir), .Y(Y));
    initial begin
        $monitor( "Time=%0t A=%b shift=%d dir=%b Y=%b", $time, A, shift, dir, Y);
        // Left shift by 0
        A = 4'b1011;
        shift = 2'b00;
        dir = 1'b0; 10;
    // Left shift by 1
        A = 4'b1011;
        shift = 2'b01;
        dir = 1'b0;
#10;
$finish ;
End
endmodule
```

4. TIMER MODULE

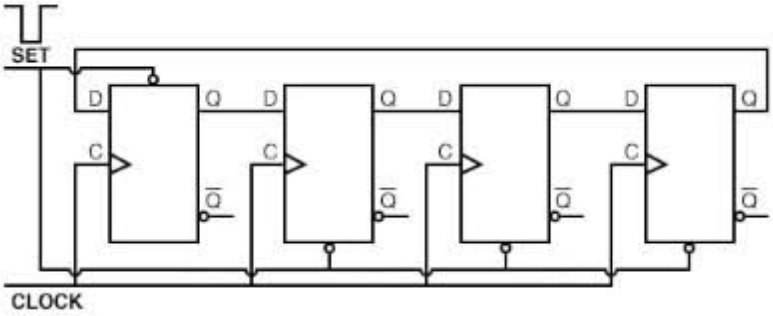
A timer module counts clock pulses to measure a specific time interval. It uses a counter that increments with each active clock edge. When the counter reaches the specified value, the timer generates an output signal. A reset signal initializes the counter to zero

HDL CODE

```
Module timer (
  input clk, reset,
  output reg done
  );
 reg [3:0] count;
  always @(posedge clk or posedge reset) begin
 if (reset) begin        count <= 0;        done <= 0;
 end
else if (count == 9) begin        count <= 0;        done <= 1;
```

```
end
  else
begin       count <= count + 1;       done <= 0;
end
 endmodule
```

5. RING COUNTER



VERILOG CODE

```
module ring_counter (
   input clk,
   input reset,
   output reg [3:0] Q
);
 always @(posedge clk or posedge reset)
   begin
      if (reset)
         Q <= 4'b0001;
      else
         Q <= {Q[2:0], Q[3]};
   End
endmodule
```

TEST BENCH

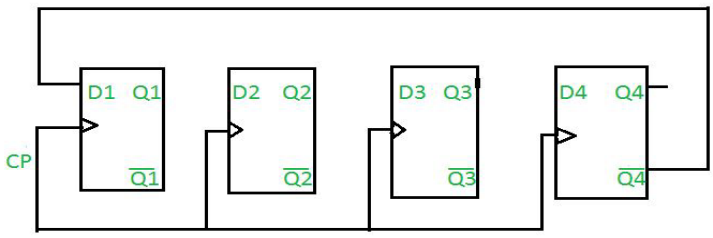
`timescale 1ns / 1ps

```
module ring_counter_tb;
reg clk;
reg reset;
```

```
wire [3:0] Q;
// Instantiate the ring counter
ring_counter uut (.clk(clk),.reset(reset),.Q(Q));
// Generate clock with 10 ns period
always #5 clk = ~clk;
initial begin
$monitor("Time=%0t reset=%b Q=%b",$time, reset, Q);
clk = 0;
reset = 1;#10;
reset = 0;
// Allow the counter to run for eight clock cycles
#80;
$finish;
end
endmodule
```

7.JOHNSON COUNTER

DESIGN



A Johnson counter is a sequential circuit in which the inverted output of the last flip-flop is connected to the input of the first flip-flop. register shifts its data on every active clock edge. An n-bit Johnson counter can generate up to 2n distinct states. It is used in sequence generation and digital control circuits.

VERILOG CODE

```
module johnson_counter (
   input clk,
   input reset,
   output reg [3:0] Q
);
always @(posedge clk or posedge reset)
  begin
      if (reset)
          Q <= 4'b0000;
       else
          Q <= {Q[2:0], ~Q[3]};
   end
endmodule
```

TEST BENCH

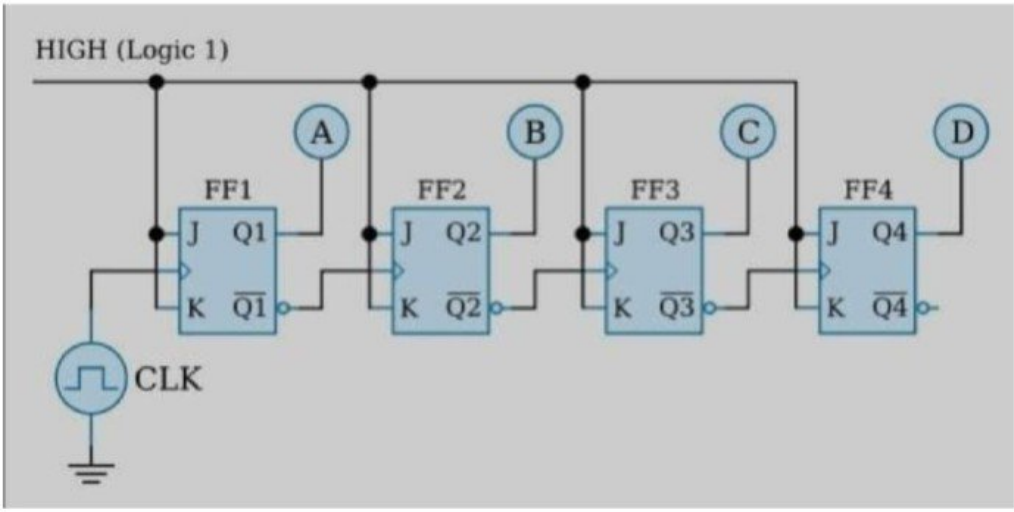
```
`timescale 1ns / 1ps
module johnson_counter_tb;
reg clk;
 reg reset;
 wire [3:0] Q;
  // Instantiate the Johnson counter
   johnson_counter uut(.clk(clk), .reset(reset),  .Q(Q) );
// Generate clock with a 10 ns period
   always #5 clk = ~clk;
 initial begin
       $monitor("Time=%0t reset=%b Q=%b, $time, reset, Q);
Clk=0;
Reset =1;
```

```
#12;
Reset=0;
#80;
// Run the counter for 8 clock cycles
  $finish;
   end
endmodule
```

8. SYNCHRONOUS COUNTER

In a synchronous counter, all flip-flops receive the same clock signal. All flip-flops change their states together at the active clock edge. It provides faster operation because the flip-flops are triggered simultaneously.

DESIGN



VERILOG CODE

```
module sync_counter (
   input clk, reset,
output reg [3:0] q

);
always @(posedge clk or posedge reset) begin
if (reset)        q <= 4'b0000;     else

      q <= q + 1'b1;
end endmodule
```

TEST BENCH:

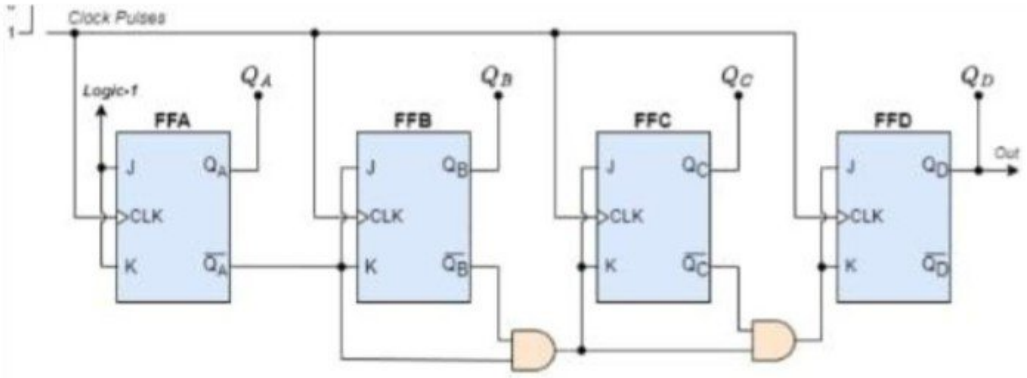
```
module tb_sync;
reg clk, reset; wire
[3:0] q;

sync_counter uut (.clk(clk), .reset(reset), .q(q));
always #5 clk = ~clk; initial begin     clk=0;
reset=1;     #10 reset=0;     #180 $finish; end
endmodule
```

9. ASYNCHRONOUS COUNTER

In an asynchronous counter, only the first flip-flop receives the external clock directly. The output of one flip-flop acts as the clock for the next flip-flop. The changes propagate through the counter, causing cumulative propagation delay.

DESIGN



VERILOG CODE

```
module async_counter (
   input clk, reset,
output reg [3:0] q

);
```

```
always @(posedge clk or posedge reset) begin
if (reset) q[0] <= 0;     else       q[0] <= ~q[0];
end

  always @(negedge q[0] or posedge reset) begin
if (reset) q[1] <= 0;     else       q[1] <= ~q[1]; end

always @(negedge q[1] or posedge reset) begin
if (reset) q[2] <= 0;     else       q[2] <= ~q[2];
end

always @(negedge q[2] or posedge reset) begin
if (reset) q[3] <= 0;     else       q[3] <= ~q[3];
end endmodule
```

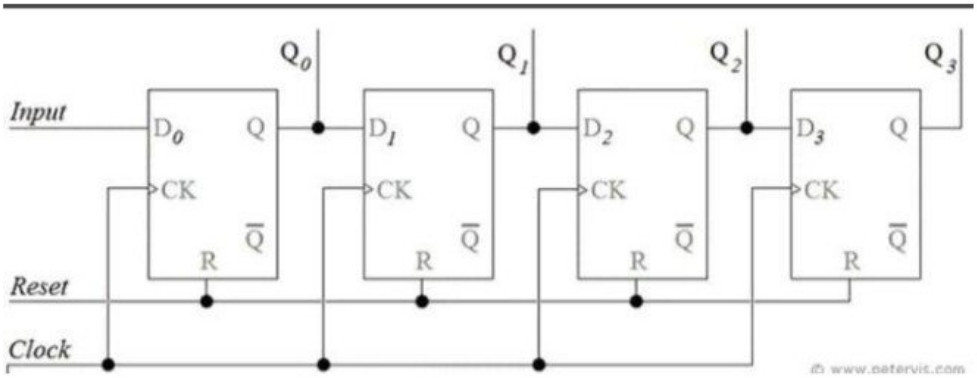
TEST BENCH:

```
module tb_async;
reg clk, reset; wire
[3:0] q;
async_counter uut
(.clk(clk),
.reset(reset),
.q(q)); always #5
clk = ~clk; initial
begin     clk=0;
reset=1;     #10
reset=0;     #180
$finish; end
endmodule
```

10.SHIFT REGISTER

A shift register is a sequential circuit used to store and shift binary data. It consists of a group of flip-flops connected in series. At every active clock edge, data shifts from one flip-flop to the next. It is used for temporary data storage and serial-to-parallel or parallel-to-serial conversion.

DESIGN



VERILOG CODE

```
module shift_register (
input clk, reset, serial_in,
output reg [3:0] q

);
always @(posedge clk or posedge reset) begin
if (reset)        q <= 4'b0000;     else

      q <= {q[2:0], serial_in};
end endmodule
```

TEST BENCH:

```
module tb_shift; reg
clk, reset, serial_in;
wire [3:0] q;
```

```
shift_register uut ( .clk(clk), .reset(reset), .serial_in(serial_in), .q(q));
always #5 clk = ~clk; initial begin
   clk=0; reset=1; serial_in=0;
   #10 reset=0;
serial_in=1; #10;
serial_in=0; #10;
serial_in=1; #10;
serial_in=1; #10;
#30 $finish; end
endmodule
```

11.DIVIDE BY N COUNTER

A divide-by-N counter divides the input clock frequency by a specified number N. It counts clock pulses from zero to N−1 and then resets to zero. A suitable output signal is generated based on the counter state. It is used in clock division and frequency control circuits.

DESIGN



VERILOG CODE

```
module divide_by_n (
   input clk, reset,
output reg clk_out
);
```

```
reg [3:0] count;
always @(posedge clk or posedge reset) begin
if (reset) begin        count <= 0;        clk_out
<= 0;    end    else if (count == 9) begin
count <= 0;        clk_out <= ~clk_out;    end
else begin        count <= count + 1'b1;    end
end
endmodule
```

TEST BENCH:

```
module tb_divide;
reg clk, reset; wire
clk_out;

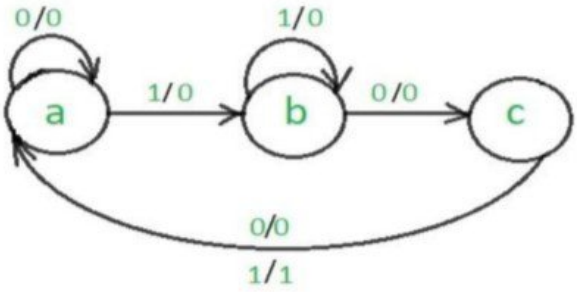
divide_by_n uut (.clk(clk), .reset(reset), .clk_out(clk_out));
always #5 clk = ~clk; initial begin    clk=0; reset=1;    #10
reset=0;    #250 $finish; end endmodule
```

12. MELAY FSM

A Mealy finite state machine produces its output based on the present state and current input. Its output can change immediately when the input changes. It generally requires fewer states than an equivalent Moore machine. It is used in sequence detectors and digital control systems.

This example detects the input sequence 101. The output becomes 1 when the final 1 completes the sequence, and the output depends on both the current state and input.

DESIGN



VERILOG CODE

```
module mealy_fsm (
   input clk, reset, din,
output reg detected

); reg [1:0] state; local param S0=0, S1=1,
S2=2; always @(posedge clk or posedge reset)
begin     if (reset) begin         state <= S0;
detected <= 0;     end     else begin
detected <= 0;         case (state)            S0: if
(din) state <= S1;                else state <= S0;
S1: if (din) state <= S1;                else state <=
S2;                    S2: if (din) begin
state <= S1;                    detected <= 1;
end               else state <= S0;  default: state
<= S0;        endcase
 end end
endmodule
```

TEST BENCH

```
module tb_mealy; reg
clk, reset, din; wire
detected;
```

```
mealy_fsm uut (.clk(clk), .reset(reset), .din(din), .detected(detected));

always #5 clk = ~clk; initial begin     clk=0; reset=1; din=0;

   #10 reset=0;

din=1; #10;     din=0;

#10;     din=1; #10;

din=0; #10;     din=1;

#10;     din=0; #10;

   $finish;

end

endmodule
```

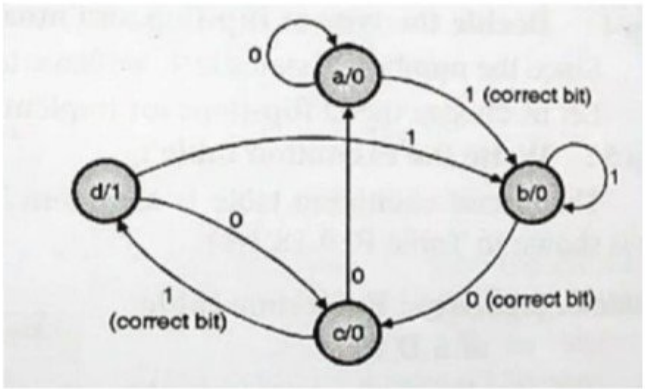
13.MOORE FSM

A Moore finite state machine produces its output based only on the present state. Its output changes when the machine transitions to another state. The output is generally more stable because it does not depend directly on the current input. It is used in controllers and sequence detection circuits.

This example also detects the sequence 101. The output becomes 1 in a dedicated detection state, so it depends only on the current state

.

DESIGN



VERILOG CODE

```
module moore_fsm (
   input clk, reset, din,
output detected

); reg [1:0]
state;

localparam S0=0, S1=1, S2=2, S3=3; assign
detected = (state == S3); always @(posedge
clk or posedge reset) begin     if (reset)
state <= S0;     else begin         case (state)

          S0: if (din) state <= S1;
else state <= S0;           S1: if
(din) state <= S1;
else state <= S2;           S2: if
(din) state <= S3;
else state <= S0;
       S3: if (din) state <= S1;
else state <= S2;

      default: state <= S0;
endcase

end

end

endmodule
```

TEST BENCH:

```
module tb_moore;
```

```
reg clk, reset, din; wire
detected;

moore_fsm uut (.clk(clk), .reset(reset), .din(din), .detected(detected));
always #5 clk = ~clk; initial begin     clk=0; reset=1; din=0;     #10
reset=0;     din=1; #10;     din=0; #10;     din=1; #10;     din=0; #10;
din=1; #10;

   #20 $finish;
end
endmodule
```

14.RANDOM SEQUENCE GENERATOR

A random sequence generator produces a sequence of binary values that appears random. It can be implemented using a Linear Feedback Shift Register (LFSR). The circuit uses shift registers and XOR feedback to generate different bit patterns. It is commonly used in testing digital circuits and generating pseudorandom sequences.

DESIGN



VERILOG CODE

```
module lfsr (
input clk, reset,
output reg [3:0] q

);
always @(posedge clk or posedge reset) begin
if (reset)        q <= 4'b0001;     else

      q <= {q[2:0], q[3] ^ q[2]};
end

endmodule
```

TEST BENCH:

```
module tb_lfsr;
reg clk, reset; wire [3:0] q; lfsr uut
(.clk(clk), .reset(reset), .q(q)); always
#5 clk = ~clk; initial begin     clk=0;
reset=1;     #10 reset=0;     #200
$finish;

 end

endmodule
```